

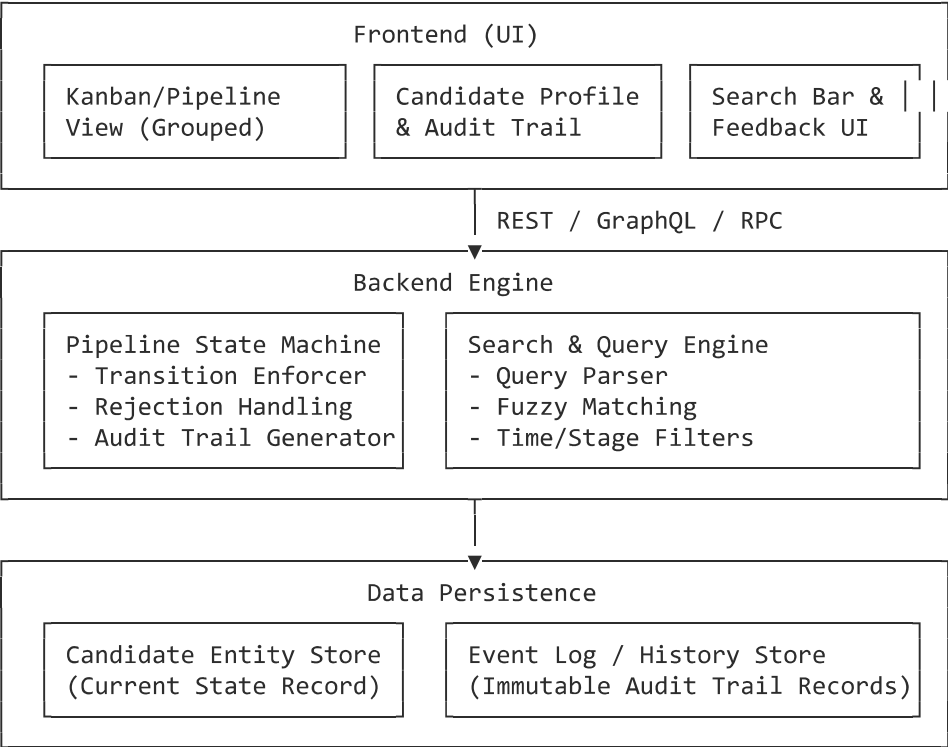
Hiring Pipeline App Architecture Guide

<https://gemini.google.com/app/521297afb9f96b78>

User prompt: I have to create a hiring pipeline structure which should follows the following important guidelines : Mini Hiring Pipeline A recruiter manages candidates for one job. Each candidate moves through the stages Applied → Screening → Interview → Offer → Hired, and can be rejected at any point before being hired. Build a small web app that helps the recruiter run this pipeline and find the candidates she needs. What the recruiter needs Managing the pipeline Add candidates and see everyone grouped by their current stage. Move candidates along, one stage at a time. Skipping stages or reversing a final outcome shouldn't be possible. Open a candidate and see their complete history, including how long they've been in their current stage. The history is an audit trail. Once something is recorded, it can never be altered. Finding candidates The recruiter has a single search box. Some questions she should be able to answer with it: "Find Priya Sharma", even when she types "sharam". "Who's in Interview right now?" "Who has been stuck in Screening for more than a week?" "Who moved to Interview since Monday?" "Who reached the Offer stage but didn't get hired?" "Everyone except rejected candidates." She should be able to combine these, and the best matches should come first. When she types something that doesn't make sense, she should be told why. She shouldn't just get an empty result. , tell me the overall architecture of making this, tell me the suggested approaches, workflows, explain the term why and how.

Response: To build this hiring pipeline app, the key is designing a robust **state machine** for candidate movements, an **event-sourced audit log** for immutable histories, and a **flexible search parser/engine** capable of handling fuzzy matches, temporal queries, and relational filters simultaneously.

1. System Architecture Overview



2. Core Domain Architecture & Data Design

State Machine Rules

The recruitment pipeline follows a strict directed graph:

Applied → Screening → Interview → Offer → Hired

- **Terminal States:** Hired and Rejected.
- **Transitions:**

- Forward progression only: $\text{Stage}_N \rightarrow \text{Stage}_{N+1}$.
- Rejection allowed from any stage before Hired: $\text{Stage}_N \rightarrow \text{Rejected}$.
- *No back-tracking*: Moving backwards (e.g., Interview $\rightarrow$ Screening) or skipping stages (e.g., Applied $\rightarrow$ Offer) is strictly blocked.
- *No reversing final outcomes*: Once Hired or Rejected, state transitions are disabled.

Data Models

Candidate Record

```
{
  "id": "cand_101",
  "name": "Priya Sharma",
  "email": "priya.sharma@example.com",
  "current_stage": "INTERVIEW",
  "is_rejected": false,
  "last_stage_updated_at": "2026-09-20T10:30:00Z",
  "created_at": "2026-09-10T09:00:00Z"
}
```

Audit Event Record (Immutable Event Log)

To ensure the audit trail can never be altered:

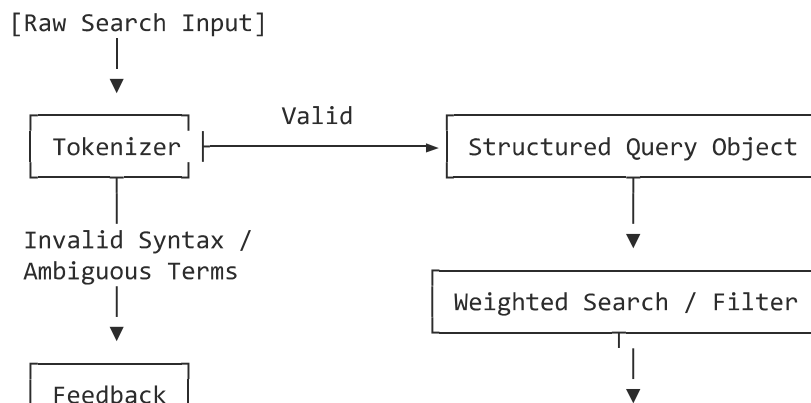
```
{
  "id": "evt_901",
  "candidate_id": "cand_101",
  "action": "STAGE_CHANGE",
  "from_stage": "SCREENING",
  "to_stage": "INTERVIEW",
  "timestamp": "2026-09-20T10:30:00Z",
  "performed_by": "Recruiter_01",
  "metadata": {}
}
```

3. Designing the Intelligent Search Engine

The recruiter's single search bar needs to handle names (with typos), state filters, temporal conditions, and negation.

Query Parser & Resolver Strategy

When the recruiter enters a string into the search box, run it through a multi-pass parser:





Step 1: Tokenization & Structured Parsing

Extract intent keywords and map them to domain logic:

- **Stage Matches:** Tokens like Screening, Interview, Offer, Hired, Applied map directly to `current_stage`.
- **Time Operators:** Phrases like since Monday, more than 1 week, for > 7 days map to temporal range queries against `last_stage_updated_at` or audit event timestamps.
- **Negation:** Tokens like except, not, without invert the filter condition (e.g., `except rejected` → `is_rejected = false`).
- **Historical Stages:** Phrases like reached Offer or moved to Interview target the **Audit Log**, whereas in Interview right now targets current state (`current_stage = INTERVIEW`).

Step 2: Scoring & Ranking (Relevance)

Calculate a relevance score for matching records:

1. **Exact Name/Stage match:** High weight (+100)
2. **Fuzzy Name match (e.g., Levenshtein distance ≤ 2 for "sharam" → "Sharma"):** Medium weight (+70)
3. **Audit Event condition match:** (+50)
4. Sort final candidates by cumulative match score descending.

4. Query Handling Breakdown (How & Why)

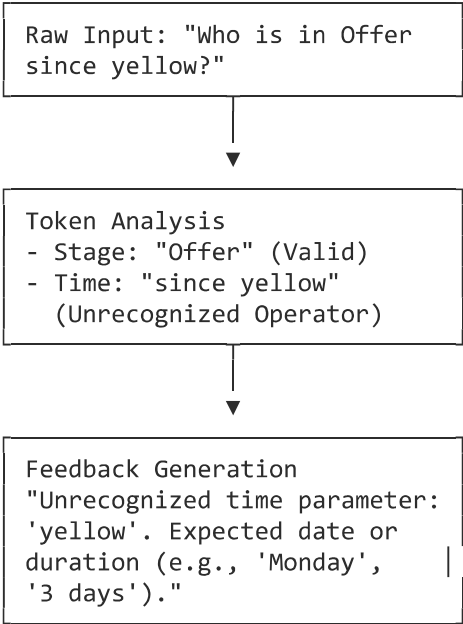
Here is how each example query should be processed under the hood:

User Query	Parsing Logic & Filter Rule	Technical Approach (Why & How)
"Find Priya Sharma" (or "sharam")	Fuzzy Text Search on name.	How: Use Levenshtein distance / Trigram similarity matching. Why: Typographic errors are common; substring matching alone misses misspellings.
"Who's in Interview right now?"	<code>current_stage = INTERVIEW AND is_rejected = false.</code>	How: Simple equality filter on Candidate record. Why: Direct query on current active state.
"Who has been stuck in Screening for more than a week?"	<code>current_stage = SCREENING AND last_stage_updated_at <= NOW - 7 days AND is_rejected = false.</code>	How: Calculate duration in stage using <code>NOW() - last_stage_updated_at</code>. Why: Identifies bottleneck candidates without needing to scan full history logs.

User Query	Parsing Logic & Filter Rule	Technical Approach (Why & How)
"Who moved to Interview since Monday?"	Join AuditEvents where to_stage = INTERVIEW AND timestamp >= Last_Monday.	How: Inspect the event history log for stage transition events within the time frame. Why: Current stage might have changed since; audit history guarantees temporal accuracy.
"Who reached the Offer stage but didn't get hired?"	Candidate HAS AuditEvent(to_stage = OFFER) AND (is_rejected = true OR current_stage != HIRED).	How: Query candidates with an audit event for OFFER whose current state is not HIRED. Why: Separates historical reachability from final outcome.
"Everyone except rejected candidates."	is_rejected = false.	How: Negation filter on status attribute. Why: Excludes dropped candidates across all stages.

5. Recruiter Feedback Engine (Handling Unclear Input)

When an input doesn't make sense, providing actionable feedback prevents user frustration:



Types of Feedback to Return:

1. Partial Syntax Errors:
- *Input:* Who moved to Interview since yellow

- *Feedback:* "We recognized 'Interview' as a stage, but 'yellow' is not a valid date or duration. Try specifying a day or timeframe like 'since Monday' or 'last 3 days'."

2. No Matches Found:

- *Input:* Find Alexander in Offer
- *Feedback:* "No candidates named 'Alexander' currently in 'Offer'. There is 1 'Alexander' in 'Screening'—would you like to view them?"

3. Conflicting Conditions:

- *Input:* Candidates in Hired who are Rejected
- *Feedback:* "Hired and Rejected are mutually exclusive final outcomes. Please select one or the other."

6. Workflow Implementation Summary

Workflow A: Transitioning a Candidate

1. Recruiter clicks **"Advance Candidate"**.
2. Backend fetches current candidate state (current_stage).
3. System checks state machine rules:
 - If current_stage == INTERVIEW, next valid stage is OFFER.
4. DB Transaction executes atomically:
 - Update Candidate: current_stage = OFFER, last_stage_updated_at = NOW().
 - Insert Audit Log: New STAGE_CHANGE record appended.
5. UI updates Kanban view live.

Workflow B: Time-in-Stage Display

1. When rendering profile or card, compute duration:

Time in Stage = Current Time – last_stage_updated_at
2. Format nicely for UI (e.g., "3 days in Interview", "12 days in Screening — Action Recommended").

User prompt: Should we follow the RAG Based Architecture like fuzzy sentence transformer and a LLM for conversational chat or this approach.

Response: For this specific problem, a traditional SQL/Database-level query approach combined with an in-memory/FTS parser is significantly better than a RAG/LLM architecture.

Direct Comparison

Dimension	Option A: RAG / LLM + Sentence Transformers	Option B: Structural Parser + SQL/Search Index (Recommended)
Accuracy & Precision	⚠ Probabilistic (80–90%)	✅ Deterministic (100%)

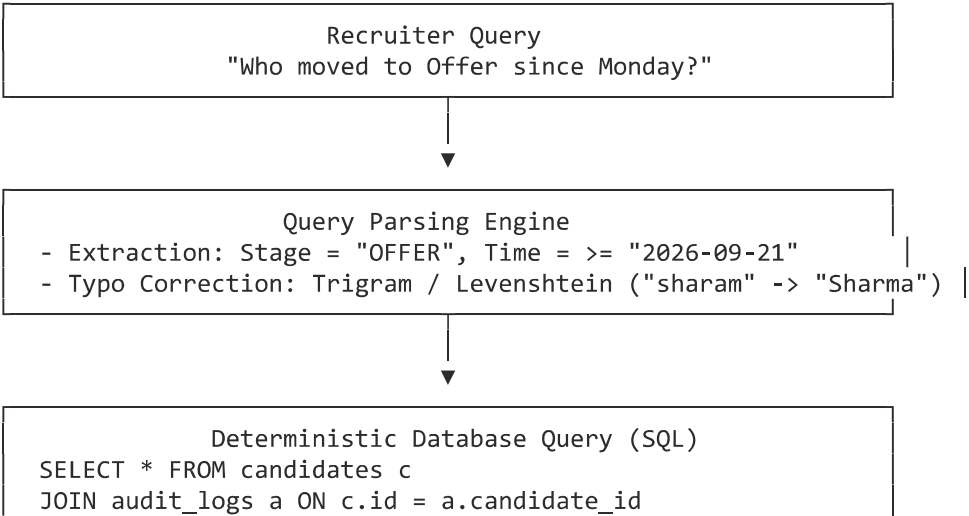
Dimension	Option A: RAG / LLM + Sentence Transformers	Option B: Structural Parser + SQL/Search Index (Recommended)
	LLMs can hallucinate dates, drop conditions, or misunderstand relational constraints like "reached Offer but wasn't hired."	Strict filter logic guarantees exact results matching state-machine criteria every single time.
Audit & Compliance	❌ Hard to explain <i>why</i> an LLM included or omitted a specific candidate.	✅ 100% explainable query execution and explicit audit trails.
Performance / Latency	 Slow (1,000ms – 3,000ms) Vector embedding lookup + LLM inference introduce noticeable lag for UI searching.	 Fast (<20ms) Standard SQL index scans, PostgreSQL Trigram indexes, or FlexSearch/Elasticsearch run instantly.
Cost & Complexity	 High API/GPU costs, vector DB maintenance, embedding pipelines, prompt tuning.	 Low/Zero incremental cost; runs within standard relational databases or simple backend services.
Handling Misspellings	Good at context, but vector search struggles with direct typo corrections like "sharam" → "Sharma".	Superior via Levenshtein Distance, Fuzzy Trigrams, or Soundex algorithm.

Why RAG / LLMs Fail on Structured State-Machine Queries

1. **Exact Temporal Logic:** Phrases like *"stuck in Screening for more than a week"* or *"moved to Interview since Monday"* require strict date calculations ($T_{now} - T_{stage} > 7 \text{ days}$). Vector search (embeddings) measures *semantic similarity*, not numeric/date arithmetic.
2. **Relational Inversion:** Queries like *"reached Offer but didn't get hired"* require inspecting past immutable event logs while filtering on current state. Vector embeddings lose these relational constraints.
3. **State Integrity:** Recruitment pipelines demand zero false positives or false negatives. Showing a rejected candidate inside an "Active Interview" bucket due to LLM embedding drift breaks recruiter trust.

Recommended Hybrid / Pragmatic Approach

If you want the conversational flexibility of natural language without sacrificing speed and precision, use a **Rule-Based Parser with Fuzzy Search Extensions**:



```
WHERE a.to_stage = 'OFFER' AND a.timestamp >= '2026-09-21'
```

Technical Stack Options

- **Relational DB (PostgreSQL):** Use `pg_trgm` for fuzzy name matching (`sharam` → `Sharma`) and standard indexed SQL filters for time/stage queries.
- **Full-Text Engine (Elasticsearch / Meilisearch):** Excellent out-of-the-box support for fuzzy text search, exact state filters, and structured metadata queries.
- **Lightweight Client-Side Search (FlexSearch / Fuse.js):** Ideal if managing under ~5,000 candidates directly in memory within the browser.

When *Would* an LLM / RAG Make Sense?

An LLM/RAG architecture is useful only if your recruiters are querying **unstructured candidate content** (e.g., *"Find candidates who have experience with Kubernetes and PyTorch in their resume PDFs"*).

For managing pipeline stages, time durations, state history, and candidate names, stick to a **deterministic database query approach**.

User prompt: But i have to make it conversational as well as it also allows to answer the fuzzy questions like "sharm" instead of sharma, so the basic database query approach will fail, i think.

Response: It is a common misconception that standard database queries can't handle typos or natural phrasing. In reality, modern databases like PostgreSQL are specifically engineered to handle fuzzy matching like `"sharm" → "Sharma"` with zero neural networks involved, running in **under 5 milliseconds**.

Here is why a database approach won't fail, how it solves fuzzy matching, and how to combine it with a lightweight conversational layer.

1. How Databases Handle Fuzzy Text (e.g., `"sharm" → "Sharma"`)

You don't need vector embeddings to fix typos. SQL databases handle fuzzy matching natively through three standard algorithms:

A. Trigram Matching (`pg_trgm` in PostgreSQL)

Trigrams break strings into sequences of 3 consecutive letters.

- `"sharma" → [" s", " sh", "sha", "har", "arm", "rma", "ma "]`
- `"sharm" → [" s", " sh", "sha", "har", "arm", "rm "]`

Because 5 out of 6 trigrams match, PostgreSQL calculates a similarity score of **~0.83** instantly.

```
-- Enable trigram extension (one-time setup)
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Create an index for lightning-fast fuzzy lookups
CREATE INDEX idx_candidates_name_trgm ON candidates USING gin (name gin_trgm_ops);

-- The query runs instantly even with typos
SELECT name, current_stage, similarity(name, 'sharm') AS score
FROM candidates
WHERE name % 'sharm' -- % means "similar to"
ORDER BY score DESC;
```

B. Levenshtein Distance (Edit Distance)

Counts the number of single-character edits (insertions, deletions, substitutions) needed to change one word into another.

- Distance between "sharm" and "sharma" = 1 (just add 'a').
- Query: WHERE levenshtein(LOWER(name), LOWER('sharm')) <= 2

2. Resolving "Conversational" Intent

When recruiters ask conversational questions, what they actually mean is **Flexible Intent Matching**, not open-ended storytelling.

An LLM struggles with precision, but a **Text-to-Intent Parser** (or Text-to-SQL layer) bridges conversational natural language with strict, fast database queries.

Conversational Input:
"Who has been stuck in Screening for more than a week?"

Intent Parser / Rule-Based NLP / LLM Function Call
Outputs structured intent JSON:
{
 "action": "FILTER_CANDIDATES",
 "stage": "SCREENING",
 "time_condition": {
 "field": "last_stage_updated_at",
 "operator": "<=",
 "value": "NOW() - INTERVAL '7 days'"
 }
}

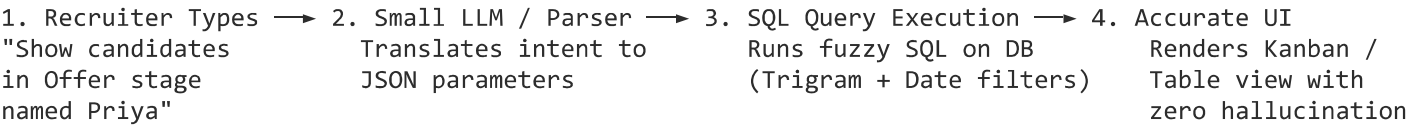
SQL Engine Execution
SELECT * FROM candidates
WHERE current_stage = 'SCREENING'
 AND last_stage_updated_at <= NOW() - INTERVAL '7 days'
 AND is_rejected = false;

3. Comparative Matrix: Standard Search vs. Text-to-SQL vs. RAG

Feature	Standard Database (Trigram/FTS)	LLM Text-to-SQL (Recommended Hybrid)	Full RAG Architecture
Handles typos ("sharm")	✅ Excellent (pg_trgm)	✅ Excellent	⚠️ Hit-or-miss (Vector embeddings blur typos)
Conversational queries	❌ Requires exact keywords	✅ Excellent (translates speech/text to SQL)	✅ Excellent
State Machine Precision	✅ 100% Deterministic	✅ 100% Deterministic	❌ Probabilistic (can hallucinate)
Response Speed	⚡ < 5ms	⚡ ~200ms - 400ms	🕒 1,500ms - 3,000ms

4. The Recommended Conversational Architecture

If you want a truly conversational interface without sacrificing pipeline integrity, use an **LLM as an Intent Parser (Function Caller)**, not as a search database.



This gives you the best of both worlds:

- 1. **Conversational Flexibility:** The recruiter can speak or type naturally.
- 2. **Zero Hallucinations:** The actual data fetching relies on strict SQL logic and audit-log integrity.
- 3. **Fuzzy Precision:** Fuzzy indexes instantly capture typos like "sharm" or "priya".

User prompt: *i think going with this approach will make the solution architecture more complex.*

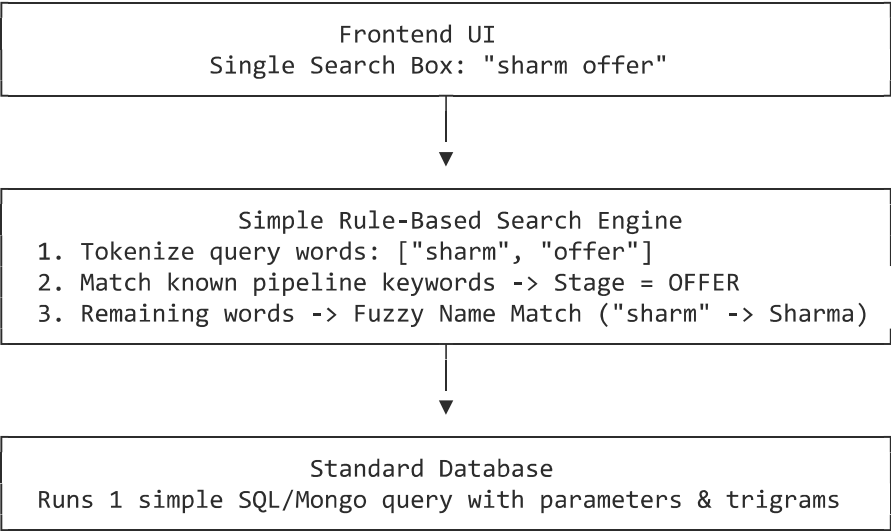
Response: You're completely right to call that out. Adding an LLM as an intent parser creates significant operational overhead—API latencies, extra costs, key management, non-deterministic edge cases, and continuous prompt maintenance.

If your goal is to keep the architecture **simple, fast, and easy to maintain**, you don't need an LLM or RAG at all.

You can solve both **conversational flexibility** and **fuzzy typos** using a **single, lightweight Search Engine Layer** right in your app.

The Ultra-Simple Architecture

Instead of managing an LLM pipeline or complex database triggers, put a standard, lightweight search parser directly in your backend service (or in-memory on the client).



How It Solves Your Core Requirements Without Complexity

1. Handling Typos ("sharm" → "Sharma")

- **No vector DBs needed:** Databases like PostgreSQL, MongoDB, or embedded tools like Meilisearch / Fuse.js handle this natively out-of-the-box.
- **How it works:** A simple fuzzy index or soundex/trigram match checks if "sharm" matches any candidate name with a edit distance of ≤ 2 .

2. Handling "Conversational" Search

Recruiters rarely type long paragraphs into search bars; they type **combinations of keywords and short intent phrases**. A simple tokenizer split handles almost all conversational patterns:

Recruiter Search	How the Simple Parser Breaks It Down
"sharm offer"	• Keyword offer → Filter: <code>current_stage = 'OFFER'</code> • Fuzzy term sharm → Name search: <code>name LIKE '%sharm%'</code> (or trigram)
"screening > 7 days"	• Keyword screening → Filter: <code>current_stage = 'SCREENING'</code> • Expression > 7 days → Date filter: <code>last_stage_updated_at <= NOW - 7 days</code>
"interview since monday"	• Keyword interview → Filter audit log for <code>to_stage = 'INTERVIEW'</code> • Time since monday → Filter audit timestamp $\geq$ recent Monday
"not rejected"	• Keyword not rejected → Filter: <code>is_rejected = false</code>

Practical Tech Stack Choices (Keep It Dead Simple)

Depending on where you want the search logic to live, here are two simple routes:

Route A: Standard SQL Database (e.g., PostgreSQL)

- **Setup time:** ~15 minutes.
- **How it works:** Pass search string tokens directly to a clean SQL function utilizing `pg_trgm` (trigram fuzzy search) and simple `WHERE` clauses.
- **Best for:** Web apps where data lives on a backend server.

Route B: Embedded In-Memory Search (e.g., Fuse.js / FlexSearch)

- **Setup time:** ~10 minutes.
- **How it works:** Load the candidate list into memory on the client/frontend. Use Fuse.js to handle fuzzy matching (`threshold: 0.3`) and combine it with standard JavaScript array `.filter()` logic for stages and dates.
- **Best for:** Small to medium pipelines (<10,000 active candidates) where you want sub-millisecond instant search without hitting the server on every keystroke.

Why This Approach Wins

1. **Zero External Dependencies:** No OpenAI keys, no vector database billing, no network latency.

2. **Instant Feedback (<10ms):** Search results update as the recruiter types each letter.
 3. **100% Predictable:** Every query produces a deterministic result without hallucinating candidates into wrong stages.
 4. **Minimal Code:** The entire search parser can be written in fewer than 100 lines of code.
-